

Python Basics, Input/Output & Variables

Input / Output Statements

- **print() Statement:** Displays output on the screen.
 - Examples: `print(10)` (printing a number), `print("ny")` (printing a string).
- **input() Function:** Receives input from the user (returns data as a string by default).
 - Examples: `x = input("Enter the number")`, `x = int(input("Enter the number"))` (casting to integer).

Identifiers & Variables

- **Identifiers:** Names given to program elements such as variables, functions, classes, or other entities.
- **Variables:** Reserved memory locations/containers that hold data values, created automatically when a value is assigned (e.g., `a = 2500`).

Data Types

- **Integer (int):** Whole numbers ranging from –negative infinity to positive infinity (e.g., -182, 182).
- **Float (float):** Numbers containing a decimal point (e.g., 3.14, 1.26).
- **String (str):** Sequence of alphabets, special characters, or numbers enclosed in quotes (e.g., "AS", "D!", "B2k!", "Anshika").
- **Boolean (bool):** Truth values represented by True or False (e.g., `10 > 5` evaluates to True).
- **List:** Built-in mutable sequence type used to store multiple items in a single variable (e.g., `G = [25, 1.27, "ADD"]`).

Identifier

Identifiers are names given to program units such as variables, functions, classes, or other entities.

Variables

Variables are like containers that hold information. A variable is created when the value is assigned to it. Example: `a=15`

Operators

Arithmetic Operator

Arithmetic operators are used in Python to do mathematical operations.

Operator	Name	Example
+	Addition	$10 + 20 = 30$
-	Subtraction	$10 - 20 = -10$
*	Multiplication	$10 * 20 = 200$
/	Division	$20 / 10 = 2$
%	Modulus	$20 \% 10 = 0$
**	Exponent	$10^{**}20 = 10^{**}20$
//	Floor Division	$9//2 = 4$

Relational Operator

Relational operators are used to check and compare values.

Name	Operator	Description
Example		

Equal to	==	Two things are exactly the same	(5 == 5)
Not equal to 5)	!=	Two things are not the same	(3 !=
Greater than 5)	>	One is greater than the other	(7 >
Less than	<	One is less than the other	(3 < 9)
Greater than or equal to	>=	Is greater or equal to an other	(8 >= 8)
Less than or equal to 6)	<=	Is less or equal to another	(4 <=

Logical Operator

Logical operators in Python are used to combine conditions and make decisions based on different situations.

Operator	Name	Example
and	AND	a and b
or	OR	a or b
not	NOT	not(a)

Conditional Statements

if Statement: Executes a block of code if a specified condition is true.

Syntax

if condition:

 #statement to be executed when the condition is true

Example:

#if statement program to check if a number is positive

```
number = int(input("Enter a number: "))
```

```
if number > 0:
```

```
    print("The number is positive.")
```

if-else Statement: Executes one block if the condition is true, and another block if it is false.

Syntax

if condition:

 #statement to be executed when the condition is true

else:

 #statement to be executed when the condition is false

Example:

Asking age to the user and providing results.

```
user_age = int(input("How old are you? "))
```

```
if user_age >= 18:
```

```
        print("You are an adult!")
else:
    print("You are a teenager or a kid.")
```

if-elif-else Statement: Evaluates multiple conditions sequentially.

Syntax

```
if condition1:
    #code to be executed if condition1 is True
elif condition2:
    #code to be executed if condition2 is True
else:
    #code to be executed when condition1 and condition2 are False
```

Example

```
# Checking the number's category, whether it is positive, negative or zero.
user_number = int(input("Enter a number: "))
if user_number > 0:
    print("The number is positive.")
elif user_number == 0:
    print("The number is zero.")
else:
    print("The number is negative.")
```

Nested if Statement: An if statement placed inside another if statement.

Syntax:

```
if condition1:
```

```
    #code to be executed if condition1 is True
```

```
    if condition:
```

```
        #code to be executed if condition2 is True
```

```
    else:
```

```
        #code to be executed when condition2 is False
```

```
else:
```

```
    #code to be executed when condition1 and condition2 are False
```

Example:

```
age = int(input("Enter your age: "))
```

```
if age >= 16:
```

```
    print("You are eligible for citizenship.")
```

```
    if age >= 18:
```

```
        print("You are eligible to cast a vote.")
```

```
    else:
```

```
        print("You are not eligible to cast vote.")
```

```
else:
```

```
    print("You are a minor.")
```

Practical Conditional Programs

```
# Smallest between two numbers
```

```
a = int(input("Enter the number: "))
b = int(input("Enter the number: "))
if a < b:
    print("a is smallest number")
else:
    print("b is smallest number")
```

Largest among three numbers

```
a = int(input("Enter the number: "))
b = int(input("Enter the number: "))
c = int(input("Enter the number: "))
if a > b and a > c:
    print("a is largest number")
elif b > a and b > c:
    print("b is largest number")
else:
    print("c is largest number")
```

Even or Odd

```
n = int(input("Enter the number: "))
if n % 2 == 0:
```

```
    print("n is even")
else:
    print("n is odd")
```

Nested IF: Voting Eligibility

```
a = int(input("Enter the age of a person: "))
if a >= 16:
    if a >= 18:
        print("can vote")
    else:
        print("can't vote")
else:
    print("not of legal age")
```

Character check (Alphabet / Digit / Special Character)

```
n = input("Enter a character: ")
if (n >= 'a' and n <= 'z') or (n >= 'A' and n <= 'Z'):
    print("Alphabet")
elif n >= '0' and n <= '9':
    print("Digit")
else:
```

```
print("Special character")
```

Check Uppercase or Lowercase Alphabet

```
n = input("Enter the alphabet: ")
```

```
if n >= 'A' and n <= 'Z':
```

```
    print("Uppercase Alphabet")
```

```
elif n >= 'a' and n <= 'z':
```

```
    print("Lowercase Alphabet")
```

```
else:
```

```
    print("It is not an alphabet")
```

Divisibility Checks

```
n = int(input("Enter the number: "))
```

```
if n % 5 == 0 and n % 11 == 0:
```

```
    print("number is divisible by 5 and 11")
```

```
else:
```

```
    print("number is not divisible by 5 and 11")
```

```
n = int(input("Enter the number: "))
```

```
if n % 5 == 0:
```

```
    print("number is divisible by 5")
```

```
elif n % 7 == 0:
```

```
    print("number is divisible by 7")
```

```
else:
```

```
    print("number is not divisible by 5 and 7")
```

Iteration (Loops)

Iteration means doing things repeatedly. Iteration is the process of repeating a particular task until a specified condition is satisfied. The most fundamental example of iteration is the “for” loop and “while” loop.

for Loop: Used for iterating over a sequence or when the number of iterations is known in advance.

Syntax:

for item in sequence:

 # Code to do something with each item

Example:

#Using for loop to print “Programming” five times

for x in range(5):

 print(“Programming”)

while Loop: Continuously executes code as long as a specified boolean condition remains True.

Syntax

while condition:

Example:

Using while loop to print numbers from 1 to 5

count = 1

while count <= 5:

 print(count)

```
count += 1
```

Python List

Python list is one of the built-in data types in Python used to store multiple data in a single variable.

Example:

```
thislist=["Computer", "Science", 20, True]
```

```
print(thislist)
```

Practical Loop Programs

Python

First 10 natural numbers using FOR loop

```
for i in range(1, 11):
```

```
    print(i)
```

Even numbers in range using FOR loop

```
for i in range(2, 21, 2):
```

```
    print(i)
```

Natural numbers using WHILE loop

```
i = 1
```

```
while i <= 10:
```

```
    print(i)
```

```
i += 1
```

Even numbers using WHILE loop

```
i = 2
```

```
n = int(input("Enter the number: "))
```

```
while i <= n:
```

```
    print(i)
```

```
    i += 2
```

Odd numbers using WHILE loop

```
i = 1
```

```
n = int(input("Enter the number: "))
```

```
while i <= n:
```

```
    print(i)
```

```
    i += 2
```

Fibonacci series using WHILE loop

```
n = int(input("Enter limit: "))
```

```
a = 0
```

```
b = 1
```

```
c = 0
```

```
while c <= n:
```

```
    c = a + b
```

```
    print(c)
```

```
    a = b
```

```
    b = c
```

7. Functions

A Python function is a block of organized, reusable code that is used to perform a single, related action. Functions provide better modularity for your application and a high degree of code reusing.

Types of Python Functions

1. Built-in functions

Python's standard library includes number of built-in functions. Some of Python's built-in functions are print(), int(), len(), sum(), etc. These functions are always available, as they are loaded into computer's memory as soon as you start Python interpreter.

2. User-defined functions

In addition to the built-in functions and functions in the built-in modules, you can also create your own functions. These functions are called user-defined functions.

Creating User Defined Function

```
def<Function name> ([parameter 1, parameter 2,...]):
```

```
    set of instructions to be executed
```

```
    [return <value>]
```

Example

Program to illustrate the use of user-defined functions

```
def add_numbers(x,y):  
    sum = x + y  
    return sum  
  
num1 = 5  
num2 = 6  
  
print("The sum is", add_numbers(num1, num2))
```

Function Classification

- **Non-Void Functions:** Functions that return a result or value using the return statement. Python supports returning multiple values separated by commas.

The computed value is returned using return statement as per syntax

```
return <value>
```

The value being returned can be one of the following:

- a. literal
- b. a variable
- c. an expression

Example

```
def sum (x., y):  
    return s
```

We are invoking this function as:

```
result = sum (5, 3)
```

Here, the return value from sum () will replace function call.

- **Void Functions:** Functions that perform actions (like printing) but do not return a value.

If a void function has a return statement, then, it takes the following form:

return

That is, keyword return without any value or expression. Following are some examples of void function:

```
def greet ( ):
```

```
    print("Hello")
```

```
def quote ( ):
```

```
    print ("python is good")
```

```
return
```

```
def greet1 ( ):
```

```
    print ("Hello", name)
```

```
def printsum (a, b, c):
```

```
    print ("Sum is, a+ b + c")
```

```
return
```

Returning Multiple Values

To return multiple values from a function, you have to ensure the return statement inside a function body should be of the form given:

```
return <value1/ variable1/ expression1>, <value2/variable2/expression2>
```

Example 1

```
print(t)
```

```
def squared(x, y, z):
```

```
    return x* x, y*y, z * z
```

```
t=squared(2,5,7)
```

Example 2

```
def squared(x, y, z):
```

```

return x* x, y*y, z * z
v1, v2, v3 = squared(2,3,4)

print("the returned values are as under:
")
print(v1 ,v2, v3)

```

Parameter & Argument Types

- **Parameters:** Parameters are the value(s) provided in the parentheses when we write function header. These are the values required by function to work.

Example

Function to calculate the area of circle using function

```
def area_of_circle (Radius):
```

```
    area = Radius** 2 * 22/7
```

```
    return area
```

```
Radius = float (input ("Please enter the radius of the given circle: "))
```

```
print (" The area of the given circle is: ", area_of_circle (Radius))
```

- **Arguments:** An argument is a value that is passed to the function when it is called. In other words, arguments are the value(s) provided in function call/invoke statement.

- **Positional / Required Arguments:** Passed in correct positional order.

For example, if a function definition header is like

```
def check (a, b, c):
```

then function calls for this can be:

```
check ( x, y, z)    # 3 values (all variables) passed
```

```
check (2, x, y)      #3 values (literal variables) passed
```

check (2, 5, 7) #3 values (all literals) passed

- **Default Arguments:** Assigns default values to parameters if none are passed.

```
def interest (principal, time, rate 0.10):
```

here 0.10 is the default value for parameter rate

The default value is specified in a manner syntactically similar to a variable initialization.

Example

```
def Interest (prin, time, rate =0.10):    #legal
```

```
def Interest (prin, time=2, rate):        #illegal(default parameter before  
required parameter
```

```
def Interest (prin =2000, time=2, rate): #illegal(default parameter before  
required parameter
```

```
def Interest (prin, time =2, rate= 0.10): #legal
```

```
def Interest (prin =200, time = 2, rate= 0.10): #legal
```

- **Keyword / Named Arguments:** Passed explicitly by parameter name.

```
interest (prin 2000, time 2, rate =0. 10)
```

```
interest (time prin = 2600, rate= 0.09)
```

```
interest (time = 2, rate = 0.12, prin =2000)
```

All the above function calls are valid now, even if the order of arguments does not match the order of parameters as defined in the function header.

Variable Scope

- **Local Scope:** Variables declared inside a function; accessible only within that function block.
- **Global Scope:** Variables declared outside any function; accessible throughout the program.

Python

Function Definition & Call

```
def addition(a, b):
```

```
    return a + b
```

```
print(addition(10, 20))
```

Void Function Example

```
def greet():
```

```
    print("Hello")
```

Returning Multiple Values

```
def calc(a, b):
```

```
    add = a + b
```

```
    sub = a - b
```

```
    return add, sub
```

```
# Function for Simple Interest
```

```
def SI(p, t, r):
```

```
    return (p * t * r) / 100
```

Library and Packages in Python

In programming, if we create smaller handleable units, called as modules. A related term is library. A library refers to a collection of modules that together supply to specific type of needs or applications e.g., NumPy library of Python provides to scientific computing needs.

Python Library

A library is a collection of modules (and packages) that together provides a specific type of applications or requirements. A library can have multiple modules in it.

Commonly used Python libraries are as listed below:

- i. NumPy:** A big library for science, it supports big arrays and lots of math functions.
- ii. Pandas:** It helps with data, offering tools for working with different kinds of data.
- iii. Matplotlib:** It's for making charts, static or interactive, and is often used with Pandas.

Python Package

A package is a container that contains various functions to perform specific tasks. Math package includes the `sqrt ( )` function to perform the square root of a number.

Importing and use of standard libraries

Python provides different ways to import and use standard libraries:

a. Import the entire module

```
import math  
  
print(math.sqrt(25))
```

Output: 5.0

You need to use `module_name.function_name()` to access functions.

b. Import a specific function

```
from math import sqrt
```

```
print(sqrt(25))
```

Output: 5.0

No need to prefix with math. when calling sqrt().

c. Import a module with an Alias (Shortcut)

```
import datetime as dt
```

```
print(dt.datetime.now()) # Prints current date and time as dt is used as a
```

shorthand for datetime.

d. Import all functions from a module

```
from math import *
```

```
print(sin(90)) # Output depends on radians
```

Output : 0.8939966636005579

Introduction to popular libraries (Math, Random, Pandas, Turtle and Matplotlib)

1. Math

Math is a built-in library in Python that provides access to mathematical functions. It includes functions for basic math operations, trigonometry, logarithms, and more.

Different mathematical functions are:

- i. Functions for mathematical constants (e.g., math.pi).
- ii. Trigonometric functions like sin(), cos(), and tan().
- iii. Logarithmic functions (log(), log10()).
- iv. Power functions like pow().

syntax.

```
import math
```

Example

```
import math

# compute the square root of 256

x = math.sqrt(256)

#assert x == 16

print("Square root is ", math.sqrt(16))

print("value is ", math.sin(math.pi/2))

print("Code works just fine, x is equal to ", x)
```

2. Random

Random function is used to generate random numbers and provide random selection. It is generally used for random sampling, generating random numbers for games, simulations, or security purposes. There are different random functions in the Random Module of Python. Some of them are:

- i. `randrange()` Returns a random number within the range
- ii. `randint()` Returns a random integer within the range
- iii. `choice()` Returns a random item from a list, tuple, or string
- iv. `random()` Generate random floating numbers

Example

```
import random

print(random.randint(1, 10))
```

```
print(random.choice(['apple', 'banana', 'Orange']))

list1 = [1, 2, 3, 4, 5, 6]

print(random.choice(list1))


import random

r1 = random.randint(5, 15)

print("Random number between 5 and 15 is % s" % (r1))

r2 = random.randint(-10, -2)

print("Random number between -10 and -2 is % d" % (r2))
```

3. Pandas

Pandas offers necessary features for effortlessly manipulating, analyzing, and managing large datasets. To achieve this, Pandas utilizes methods like data frame operations, time series analysis, and data cleansing.

\$ pip install pandas

pandas.read_csv() Loads data from a CSV file into a table-like structure called a DataFrame.

DataFrame.info() Shows a summary of the DataFrame: column names, data types, and missing values.

DataFrame.shape Returns the size of the DataFrame as a tuple (rows, columns).

pandas.DataFrame() Creates a table (DataFrame) with rows and columns that can hold different types of data

Example

```
import pandas as pd

data = {'Name': ['Shyam', 'Sanskar'], 'Age': [25, 30]}

df = pd.DataFrame(data)

print(df)
```

4. Turtle

The turtle module in Python is a pre-built module in Python that can be used to create a graphical representation on the terminal using a cursor which is known as a turtle.

Syntax

```
import turtle
```

Example

```
import turtle

s = turtle.Turtle()

for i in range(4):

    s.forward(50)

    s.right(90)

turtle.done()
```

5. matplotlib

Matplotlib is a low-level graph plotting library in python that serves as a visualization utility for high-quality visualizations and graphs.

Before using we should install Matplotlib by using the PIP command.

pip install matplotlib

There are different types of plots in Matplotlib. Some of them are:

- i. Line plot: A line plot shows the relationship between the x and y-axis.
- ii. Bar Plot: the barplot shows the relationship between the numeric and categoric values
- iii. Pie charts: A pie chart (or circular chart) is used to show the percentage of the whole.

Graphics Using Turtle

Define, draw turtle, function (Forward, Backward, Left, Right, Penup, color, Fillcolor), shape. The turtle module in Python is a pre-built module in Python that can be used to create a graphical representation on the terminal using a cursor which is known as a turtle.

Use of Turtle Module

- i. Easy visualization of programming concepts
- ii. Interactive learning
- iii. Enhances creativity
- iv. Simplified debugging

Turtle Motion

a) forward(distance) or turtle.fd(distance)

It moves the turtle in the forward direction by a certain distance.

Example

```
import turtle
```

```
sk = turtle.Turtle()
```

```
sk.forward(50)
```

```
turtle.done()
```

b) back(distance) or turtle.bk or turtle.backward(distance)

This method moves the turtle in the opposite direction the turtle is headed.

Example:

```
import turtle
```

```
sk = turtle.Turtle()
```

```
sk.backward(50)
```

```
turtle.mainloop()
```

c) right(angle) or turtle.rt(angle)

This method moves the turtle right by angle units.

Example:

```
import turtle
```

```
t = turtle.Turtle()
```

```
t.heading()
```

```
t.right(30)
```

```
t.heading()
```

```
turtle.mainloop()
```

d) left(angle) or turtle.lt(angle) –

This method turns the turtle left by angle units.

Example:

```
import turtle  
  
t = turtle.Turtle()  
  
t.heading()  
  
t.left(100)  
  
t.heading()  
  
turtle.mainloop()
```

e) penup()

This will lift the turtle off the “digital canvas” and if you move the turtle in penup state it won’t draw.

Example:

```
import turtle  
  
turtle.color("red")  
  
turtle.hideturtle()
```

```
turtle.speed(1)
turtle.left(90)
for i in range(4):
    turtle.forward(30)
    turtle.penup()
    turtle.forward(30)
    turtle.pendown()
turtle.exitonclick()
```

f) color() :

This method is used to change the color of the ink of the turtle drawing. The default color is black.

Syntax:

```
turtle.color(*args)
```

Example:

```
import turtle
turtle.forward(50)
# change the color of turtle
turtle.color("blue")
```

```
turtle.forward(150)
```

```
turtle.color("red")
```

```
turtle.forward(50)
```

g) fillcolor():

This helps to choose the color for filling the shape. It takes the input parameter as the color name or hex value of the color and fills the upcoming closed geographical objects with the chosen color. We need following to implement it

begin_fill(): This function tells turtle that all upcoming closed graphical objects needed to be filled by the chosen color.

end_fill(): This function tells turtle to stop the filling upcoming closed graphical objects.

Example:

```
import turtle
```

```
# creating turtle pen
```

```
t = turtle.Turtle()
```

```
# taking input for the radius of the circle
```

```
r = int(input("Enter the radius of the circle: "))
```

```
# taking the input for the color
```

```
col = input("Enter the color name or hex value of color(# RRGGBB): ")
```

```
# set the fillcolor

t.fillcolor(col)

# start the filling color

t.begin_fill()

# drawing the circle of radius r

t.circle(r)

# ending the filling of the color

t.end_fill()
```

Output:

h) shape()

This function is used to set the turtle shape to shape with a given name or, if the name is not given, return the name of the current shape.

Syntax:

```
turtle.shape(name=None)
```

Polygon shapes available in Python are: “arrow”, “turtle”, “circle”, “square”, “triangle”, “classic”.

Example:

```
import turtle
```

```
# for default shape
turtle.forward(100)

# for circle shape
turtle.shape("circle")
turtle.right(60)
turtle.forward(100)

# for triangle shape
turtle.shape("triangle")
turtle.right(60)
turtle.forward(100)

# for square shape
turtle.shape("square")
turtle.right(60)
turtle.forward(100)

# for arrow shape
turtle.shape("arrow")
turtle.right(60)
turtle.forward(100)

# for turtle shape
turtle.shape("turtle")
turtle.right(60)
```

```
turtle.forward(100)
```

Error Handling & Exception Handling

Errors: An error is an issue in a program that prevents the program from completing its task. Problems in code that can cause it to fail.

Types of Errors

- **Syntax Error:** Code violates language syntax rules and fails prior to parsing/compilation.
- **Runtime Error:** Errors that occur during program execution (e.g., division by zero, invalid index).
- **Logical Error:** The program runs smoothly without throwing errors, but yields incorrect outputs due to bad logic.

Exceptions:

Even if a statement or expression is syntactically correct, it may cause an error when an attempt is made to execute it. Errors detected during execution are called exceptions and are not unconditionally fatal.

Exception Handling Blocks

- **try Block:** Encloses code that might potentially raise an exception.
- **except Block:** Handles specific exceptions raised inside the try block.
- **else Block:** Executes if no exceptions occur in the try block.

- **finally Block:** Always executes regardless of whether an exception was raised or handled.

Python

```
num1 = int(input("Enter num1: "))
```

```
num2 = int(input("Enter num2: "))
```

```
try:
```

```
    res = num1 / num2
```

```
except ZeroDivisionError:
```

```
    print("Can't divide by zero")
```

```
except ValueError:
```

```
    print("Invalid number format")
```

```
else:
```

```
    print("The result is:", res)
```

```
finally:
```

```
    print("Execution complete.")
```

File Handling

File handling in Python involves interacting with files on your computer to read data from them or write data to them. The pandas library in Python is a powerful tool for data manipulation and analysis.

Benefits of file handling in Python

- i. Versatility
- ii. Flexibility
- iii. User friendly
- iv. Cross-platform

Difficulties of file handling in Python

- i. Error-prone
- ii. Security risks
- iii. Complexity
- iv. Performance

Opening a file in Python

To perform any file operation, the first step is to open the file. Python's built-in `open()` function is used to open files in various modes, such as reading, writing, and appending.

Syntax : with `open('Filename', 'mode')` as file:

File Access Modes

- **'r' (Read):** Opens file for reading only. Raises an error if the file does not exist.

- **'w' (Write):** Opens file for writing. Overwrites existing file content or creates a new file.
- **'a' (Append):** Opens file for appending new data to the end without overwriting existing contents.
- **'x' (Exclusive Creation):** Creates a new file; fails if the file already exists.

File Operations Code

Reading a file

Reading a file can be achieved by `file.read()` which reads the entire content of the file. We can close the file using `file.close()` which closes the file after reading it, which is necessary to free up system resources.

Example:

```
file = open("Test.txt", "r")  
  
content = file.read()  
  
print(content)  
  
file.close()
```

Methods of reading

1. `readline ()`
2. `readlines()`
3. `read(size)`

Example:

#read a file

nl = open('nl.txt','r') # it will open the file in read mode

nl.readline() # it will read the first line of the file

nl.read(10) # it will read the first 10 character of the file

nl.read() # it will return the whole text

Create a new file in Python

To create a new file, we have to open a file using one of the two parameters:

x: It will create a new empty text file if there does not exist any file with the same name; otherwise, it will throw an error

w: It will create a file, whether any file exists with the same or not, i.e., it will not return an error if the same file name exists.

Example:

#create a file

nl = open ('nl.txt', 'x') # It will create a new empty file but will throw an error if the same file name exists

nl = open ('nl.txt', 'w') # It will create a file but if the same file name exist it will overwrite

Writing strings to a text file

Writing to a file is done using `file.write()` which writes the specified string to the file.

```
file = open('xyz.txt', 'w')
```

```
file.write('Hello\n')
```

```
file.write('My name\n')
```

```
file.write('is Ravi\n')
```

```
file.close()
```

methods to write in an existing file:

```
file = open('xyz.txt', 'w')
```

```
file.writelines(['Hello\n', 'My name\n', 'is Ravi\n'])
```

```
file.close()
```

(a) write () method

It will add a single line at a time

Example:

```
with open('file.txt','w') as nl:
```

```
nl.write('This is the first line\n')
```

```
nl.write('This is the second line\n')
```

```
nl.write('this is the third line')
```

(b) writelines () method

It allows to insert multiple string at a time

Example:

```
nl = open('file.txt', 'w+')
```

#It allows to write all the line in one go

```
nl.writelines(['This is the first line\n', 'This is the second line\n', 'This is the third line\n'])
```

```
nl.close()
```

Example:

Python code to illustrate append() mode

```
file = open('geek.txt', 'a')
```

```
file.write("Python programming isfun")
```

```
file.close()
```

Reading CSV files using standard csv module

```
import csv
```

```
with open('xyz.csv', 'r') as csvfile:
```

```
    reader = csv.reader(csvfile)
```

```
    for row in reader:
```

```
print(row)
```

Closing a file in Python

Once you are done with all the operations, you need to close the file correctly.

The close () commands will terminate all the resources and will free the system

It is a good practice to close the file.

Example:

```
#close a file
```

```
nl = open('nl.txt', 'r')
```

```
print(nl.read())
```

```
nl.close()
```

Read and write CSV file using panda

CSV (Comma Separated Values) format is the most common import and export format for spreadsheets and databases. The structure of a CSV file is given away by its name.

Structure is as shown below:

column 1 name, column 2 name, column 3 name

first row data 1, first row data 2, first row data 3

second row data 1, second row data 2, second row data 3

CSV files are favoured for a few key reasons:

a) Portability

b) Simplicity

c) Wide support

Benefit of the csv Module

a) Built-in and easy to

b) Flexible and adaptable

c) Memory efficient

Reading CSV

Reading from a CSV file is done using the reader object. The CSV file is opened as a text file with Python's built-in open() function, which returns a file object.

There are two main ways to read CSV files in Python:

a) Using the csv module: This is the built-in module for working with CSV files in Python. It provides basic functionality for reading and writing CSV data.

Example using csv.reader:

```
import csv

# Open the CSV file in read mode
with open('data.csv', 'r') as csvfile:

    # Create a reader object
    csv_reader = csv.reader(csvfile)

    # Iterate through the rows in the CSV file
    for row in csv_reader:

        # Access each element in the row print(row)
```

b) Using the Pandas library: Pandas is a powerful library for data analysis in Python. It offers a more convenient way to read and manipulate CSV data. We should use following syntax to read file using panda library

Step1: Import pandas library

```
import pandas as pd
```

Step2: Load CSV files to pandas using read_csv()

Basic Syntax: `pandas.read_csv(filename, delimiter=',')`

```
data= pd.read_csv("Salary_Data.csv")
```

Years	Experience	Salary
0	1.1	39343.0
1	1.3	46205.0
2	1.5	37731.0
3	2.0	43525.0
4	2.2	39891.0
5	2.9	56642.0
6	3.0	60150.0
7	3.2	54445.0
8	3.2	64445.0
9	3.7	57189.0

10 3.9 63218.0

Step3: Extract the field names.

.columns is used to obtain the header/field names.

```
data.columns
```

Step4: Extract the rows.

All the data of a data frame can be accessed using the field names.

```
data.Salary
```

Example:

```
import pandas as pd  
  
# Reading the CSV file  
  
df = pd.read_csv('Data.csv')  
  
# Displaying the content  
  
print(df)
```

Write to a CSV file

The DataFrame supports the to_csv() method used to write a CSV file.

We need to follow these steps to write to a CSV file using pandas:

Step1: Import pandas library

```
import pandas as pd
```

Step2: Create a pandas dataframe using pd.DataFrame

Syntax: `pd.DataFrame(data, columns)`

The data parameter takes the records/observations, and the columns parameter takes the columns/field names.

```
header = ['Name', 'M1 Score', 'M2 Score']
```

```
data = [['Sanskar', 62, 80], ['Sambriddhi', 45, 56], ['Aurora', 85, 98]]
```

```
data = pd.DataFrame(data, columns=header)
```

Step3: Write to a CSV file using to_csv()

Syntax: `DataFrame.to_csv(filename, sep=',', index=False)`

****separator is ',' by default.**

`index=False` to remove the index numbers.

```
data.to_csv('Stu_data.csv', index=False)
```

Example

```
import pandas as pd
```

```
# Data to be written
```

```
data = {
```

```
    'Product': ['Laptop', 'Smartphone', 'Tablet'],
```

```
    'Price': [75000, 15000, 20000],
```

```
    'Quantity': [3, 10, 5]
```

```
}  
  
# Creating a DataFrame  
  
df = pd.DataFrame(data)  
  
# Writing the DataFrame to a CSV file  
  
df.to_csv('products.csv', index=False)  
  
print("Data written to products.csv successfully.")
```

Data Visualization

Data visualization means showing data using charts or graphs to understand it better. It helps us see patterns, trends, and connections in a simple way.

Some popular Python libraries are:

- i. Matplotlib – Good for simple graphs like bar charts and line graphs.
- ii. Seaborn – Makes more beautiful and colorful charts easily.
- iii. Plotly – Used for interactive and dynamic graphs.

Matplotlib

Matplotlib is the most popular Python plotting library.

To install Matplotlib, pip, and conda can be used, and should install using command prompt.

```
pip install matplotlib
```

or

```
conda install matplotlib
```

Matplotlib is specifically suitable for creating basic graphs like line charts, bar charts, histograms, etc. It can be imported by typing:

```
import matplotlib.pyplot as plt
```

1. Scatter plot

Example:

```
#Importing the libraries
```

```
import pandas as pda
```

```
import matplotlib.pyplot as plt
```

```
# Reading the dataset
```

```
dataset = pda.read_csv("Stu_data.csv")
```

```
plt.scatter(dataset['Name'],
```

```
dataset['Marks'])
```

```
plt.title("Scatter Plot")
```

```
plt.xlabel('Name')
```

```
plt.ylabel('Marks')
```

```
plt.show()
```

2. Bar chart

To represent a data category, a bar plot or bar chart uses rectangular bars whose lengths and heights correspond to the data values they represent.

Example:

```
# Importing the libraries
```

```
import pandas as pda
```

```
import matplotlib.pyplot as plt
```

```
# Reading the database
```

```
data = pda.read_csv("tips.csv")
```

```
plt.bar(data['total_bill'], data['day'])
```

```
plt.title("Bar Chart")
```

```
plt.xlabel('Day')
```

```
plt.ylabel('Tip')
```

```
plt.show()
```

Seaborn

Seaborn is a Python library used to make beautiful and informative charts. It is built on top of Matplotlib and works well with Pandas data.

Seaborn is great for creating charts that show patterns and relationships in data, such as:

i. Line plots

ii. Bar plots

iii. Heatmaps

To install Seaborn, use this command in the command prompt:

```
pip install Seaborn
```

It works best in Jupyter Notebook or IPython, which show the charts more clearly.

1. Line Plot

To plot a line plot in Seaborn, use the `lineplot()` method. It is also possible to pass just the data argument in this case.

Example:

```
# Importing the libraries
```

```
import pandas as pda
```

```
import seaborn as sn
```

```
import matplotlib.pyplot as plt
```

```
# Reading the database
```

```
dataset = pda.read_
```

```
csv("Stu_data.csv")
```

```
sn.lineplot(x='Name',
```

```
y='Marks', data=dataset)
```

```
plt.show()
```

Plotly

Plotly is an open-source Python library used to make interactive charts. It can create different types of graphs like:

- i. 3D charts
- ii. Scientific and statistical charts
- iii. Financial charts

To use Plotly we should install it by using following command

```
pip install plotly
```

1. Scatter Plot Plotly's scatter() method can be used to create scatter plots. It is also necessary to include an additional data argument, like Seaborn. T

```
import pandas as pda
```

```
import plotly.express
```

```
# we are reading the csv dataset through pandas
```

```
dataset = pda.read_csv("tips.csv")
```

```
graph = plotly.express.scatter(dataset, x="total_bill", y="size", color='smoker')
```

```
graph.show()
```

2. Line chart

In Plotly, line plots are much more accessible and illustrious additions that assemble easy-to-style statistics from various data types.

Example:

```
# importing the library

import plotly.express as px

import pandas as pda

# we are reading the database

data = pda.read_csv("stu_data. csv")

# we are plotting the scatter chart

fig = px.line(data, y='Name', color='Gender')

fig.show()
```

3. Bar chart

With plotly.express, you can create bar charts using the bar() method.

```
# importing the libraries

import plotly.express as px

import pandas as pd

# reading the database

data = pd.read_csv("Stu_data.csv")

fig = px.bar(data, x='Name', y='Marks', color='Gender')

# showing the plot
```

```
fig.show()
```